MODULE 2 ASSIGNMENT

Lab Exercise

1)Research and provide three real-world applications where C programming is extensively used, such as in embedded systems, operating systems, or game development.

Embedded Systems – Used in devices like cars (ECUs), microwaves, cameras; C gives low-level hardware control and efficiency.

Operating Systems – Linux, Windows, MacOS use C for kernels and system-level code due to speed and portability.

Game Development – High-performance games and engines (Unreal Engine, OpenGL) use C for fast execution and memory control.

Theory Exercise

1) History and Evolution of C Programming

- C programming language was developed in 1972 by Dennis Ritchie at Bell Laboratories.
- It was created mainly for developing the UNIX operating system.
- C was developed from the B language and improved its limitations.
- In 1978, the book The C Programming Language by Kernighan and Ritchie helped popularize C.
- Later, C was standardized by ANSI, leading to versions like C89, C99, and C11.

Importance of C Programming

- C is a powerful and efficient programming language.
- It helps programmers understand memory management, pointers, and system-level concepts.
- Many modern languages like C++, Java, and Python are influenced by C.
- C programs are portable and can run on different machines with minor changes.

Why C Is Still Used Today

- C provides high performance and speed.

- It is widely used in operating systems, embedded systems, compilers, and device drivers.
- C requires less memory, making it suitable for hardware-related applications.
- It builds a strong programming foundation for students and developers.

2) Describe the steps to install a C compiler (e.g., GCC) and set up an Integrated Development Environment (IDE) like DevC++, VS Code, or CodeBlocks.

Installing a C Compiler (GCC)

First, download a C compiler such as GCC (GNU Compiler Collection).

On Windows, GCC is usually installed using MinGW or MinGW-w64.

Download MinGW from its official website and run the installer.

During installation, select gcc and complete the setup.

After installation, set the PATH environment variable so that the compiler can be used from the command line.

To verify installation, open Command Prompt and type gcc --version.

2. Setting Up Dev-C++

Download Dev-C++ from its official website.

Run the installer and follow the on-screen instructions.

Dev-C++ comes with a built-in compiler, so no separate setup is usually required.

Open Dev-C++, create a new C file, write code, and compile/run it using the Run option.

3. Setting Up CodeBlocks

Download CodeBlocks (with MinGW version) from the official website.

Install it by following the setup wizard.

During first launch, CodeBlocks automatically detects the compiler.

Create a new console application project, write C code, and build/run it.

4. Setting Up VS Code for C Programming

Download and install Visual Studio Code.

Install the C/C++ extension from the Extensions Marketplace.

Make sure GCC is already installed on the system.

Configure the compiler path in VS Code settings if required.

Create a .c file and use the terminal in VS Code to compile and run the program.

3) Explain the basic structure of a C program, including headers, main function, comments, data types, and variables. Provide examples.

Header Files:

Header files contain declarations of predefined functions, macros, and constants. They are included at the beginning of a C program using the #include directive. Header files help in performing tasks like input, output, mathematical operations, and string handling.

Preprocessor Directives:

Preprocessor directives are instructions given to the compiler before actual compilation starts. They begin with the # symbol and are used for including header files, defining constants, and macros.

main() Function:

The main() function is the entry point of a C program. Program execution always starts from the main() function. It contains the logic of the program and returns an integer value to indicate successful or unsuccessful execution.

Comments:

Comments are used to explain the program logic and make the code more readable. They do not affect program execution because the compiler ignores them. C supports single-line and multi-line comments.

Data Types:

Data types define the type and size of data that a variable can store. They help in proper memory allocation and data handling. Common data types include integer, character, floating-point, and double types.

Variables:

Variables are named memory locations used to store data values during program execution. Each variable must be declared with a data type before it is used in the program.

Statements and Expressions:

Statements are instructions given to the computer to perform specific tasks such as calculations, input, and output. Expressions are combinations of variables, constants, and operators.

Return Statement:

The return statement is used to send a value from the main() function back to the operating system. It indicates the termination of the program.

4) Write notes explaining each type of operator in C: arithmetic, relational,logical, assignment, increment/decrement, bitwise, and conditional operators.

Arithmetic Operators

Arithmetic operators are used to perform basic mathematical calculations such as addition, subtraction, multiplication, division, and modulus. These operators work on numeric data types and are commonly used in expressions and calculations.

2. Relational Operators

Relational operators are used to compare two values. They check relationships such as greater than, less than, or equality. The result of a relational operation is either true or false, which is mainly used in decision-making statements.

3. Logical Operators

Logical operators are used to combine two or more conditions or to reverse a condition. They return true or false based on the logical relationship between expressions. These operators are mostly used in conditional statements and loops.

4. Assignment Operators

Assignment operators are used to assign values to variables. They can also perform an operation and assignment in a single step. These operators help reduce code length and improve readability.

5. Increment and Decrement Operators

Increment and decrement operators are used to increase or decrease the value of a variable by one. They are frequently used in loops and counters. These operators can be applied before or after a variable.

6. Bitwise Operators

Bitwise operators perform operations at the bit level of integer data. They directly manipulate binary digits (bits) of a number and are mainly used in low-level programming, embedded systems, and hardware-related applications.

7. Conditional (Ternary) Operator

The conditional operator is a special operator that works as a shorthand for the if-else statement. It evaluates a condition and returns one value if the condition is true and another value if the condition is false.

5) Explain decision-making statements in C (if, else, nested if-else, switch).

Provide examples of each.

if Statement

The if statement is used to execute a block of code only when a given condition is true.

Example:

```
if (age >= 18)
{
   printf("Eligible to vote");
}
```

2. if–else Statement

The if–else statement is used when there are two possible conditions. If the condition is true, the if block executes; otherwise, the else block executes.

Example:

```
if (marks >= 40)
{
   printf("Pass");
}
else{
   printf("Fail");
}
```

3. Nested if–else Statement

A nested if–else statement is an if–else inside another if or else. It is used when multiple conditions need to be checked.

Example:

```
if (marks >= 60)
{
   printf("First Class");
}
else if (marks >= 40)
{
   printf("Second Class");
}
else
{
   printf("Fail");
}
```

4. switch Statement

The switch statement is used to select one block of code from multiple options based on the value of an expression. It is mainly used instead of multiple if–else statements when checking a single variable.

Example:

```
switch(choice)
{
   case 1:
      printf("Pizza");
      break;
   case 2:
      printf("Burger");
      break;
```

```
    case 3:
        printf("Coffee");
        break;
    default:
        printf("Invalid choice");
}
```

6) Compare and contrast while loops, for loops, and do-while loops. Explain the scenarios in which each loop is most appropriate.

Feature	**for loop**	**while loop**	**do while loop**
type of Control	Entry-controlled	Entry-controlled	Exit-controlled
Condition Check	Before execution	Before execution	After execution
Minumum Execution	May execute 0 times	May execute 0 times	Executes at least once
Syntax Structure	Initialization done separately	Initialization, condition & increment in one line	Condition written at the end
Best Used When	Iterations unknown	Iterations known	Body must run at least once

- **while loop:** Used when number of repetitions is not known and depends on a condition.
  Scenario: Taking user input until user enters 0.

- **for loop:** Used when number of repetitions is known in advance.
  Scenario: Printing numbers from 1 to 10 or printing pattern 5 times.

- **do-while loop:** Used when loop must execute at least one time, even if condition is false.
  Scenario: Showing menu once and asking user whether to continue or exit.

7) Explain the use of break, continue, and goto statements in C. Provide

examples of each.

1. break Statement

Explanation:

The break statement is used to immediately exit from a loop or switch statement. When break is executed, the control moves to the statement after the loop.

Example:

```
#include<stdio.h>
int main()
{
   for(int i=1; i<=10; i++)
   {
      if(i==5){
         break;
      }
      printf("%d ", i); }
   return 0;
}
```

Output:

1 2 3 4

Use:

Used when we want to stop the loop before it finishes.

2. continue Statement

Explanation:

The continue statement is used to skip the current iteration and move to the next iteration of the loop.

Example:

```
#include<stdio.h>
int main()
```

```
{
   for(int i=1; i<=5; i++)
   {
      if(i==3)
      {
         continue;
      }
      printf("%d ", i);
   }
   return 0;
}
```

Output:

1 2 4 5

Use:

Used when we want to skip a specific value.

3. goto Statement

Explanation:

The goto statement is used to transfer control to another part of the program using a label.

Example:

```
#include<stdio.h>
int main()
{
   int i=1;

start:
   if(i<=5)
```

```
    {
        printf("%d ", i);
        i++;
        goto start;
    }

    return 0;
}
```

Output:

1 2 3 4 5

Use:

Used to jump to a specific label in the program.

8) What are functions in C? Explain function declaration, definition, and how to call a function. Provide examples.

A function in C is a block of code that performs a specific task.

Functions help divide a large program into smaller, manageable parts.

They improve code reusability, readability, and modularity.

Types of Functions in C:

Library Functions

These are predefined functions provided by C libraries.

Examples: printf(), scanf(), sqrt(), strlen().

User-Defined Functions

These are functions created by the programmer to perform specific tasks.

Three Parts of a Function

A function in C consists of three main parts:

1. Function Declaration (Prototype)

The function declaration informs the compiler about:

Function name

Return type

Number and type of parameters

It is written before the main() function.

Example:

```
int add(int, int);
```

2. Function Definition

The function definition contains the actual body of the function.

It specifies what the function does.

Example:

```
int add(int a, int b)
{
   return a + b;
}
```

3. Function Call

A function call is used to execute the function.

It passes arguments to the function.

Syntax:

Example:

```
result = add(10, 20);
```

9) Explain the concept of arrays in C. Differentiate between one-dimensional and multi-dimensional arrays with examples.

An array in C is a collection of elements of the same data type stored in contiguous memory locations. Arrays are used to store multiple values in a single variable instead of declaring separate variables for each value.

One-Dimensional Array	Multi-Dimensional Array
Stores data in a single row	Stores data in rows and columns
Uses one index	Uses two or more indexes
List format	Table or matrix format
Marks of students	Marks of students in different subjects

10) Explain what pointers are in C and how they are declared and initialized. Why are pointers important in C?

A pointer in C is a special type of variable that stores the memory address of another variable. Instead of storing a direct value, a pointer stores the location where the value is stored

Declaration of Pointers

A pointer is declared by specifying the data type it will point to, followed by an asterisk (*) symbol and the pointer name.

The data type indicates the type of variable whose address will be stored in the pointer.

For example, if a pointer is declared for an integer type, it can store the address of an integer variable.

Initialization of Pointers

Initialization means assigning the address of a variable to the pointer.

This is done using the address operator (&), which gives the memory address of a variable.

After initialization:

The normal variable stores the actual value.

The pointer stores the memory address of that variable.

Importance of Pointers in C

- Pointers allow dynamic memory allocation during program execution.
- They help in passing arguments to functions by reference.
- Pointers make memory usage more efficient.

- They are essential for implementing data structures like linked lists, stacks, queues, and trees.
- Pointers provide direct access to memory locations.
- They improve program performance by reducing memory copying.

11) Explain string handling functions like strlen(), strcpy(), strcat(), strcmp(), and strchr(). Provide examples of when these functions are useful.

1. strlen()

It is used to find the length of a string.

printf("%d", strlen("Hi"));

Output: 2

2. strcpy()

It is used to copy one string into another.

strcpy(str1, "Raj");

Now str1 contains "Raj".

3. strcat()

It is used to join two strings.

strcat(str1, str2);

If str1 = "Hi " and str2 = "Raj"

Result → "Hi Raj"

4. strcmp()

It is used to compare two strings.

strcmp("Hi", "Hi");

Returns 0 (Strings are equal)

5. strchr()

It is used to find a character in a string.

strchr("Rajvi", 'v');

Finds character 'v'.

12) Explain the concept of structures in C. Describe how to declare, initialize, and access structure members.

A structure in C is a user-defined data type that allows us to combine different types of data into a single unit. Unlike arrays, which store elements of the same data type, structures can store elements of different data types.

Structures are useful when we want to store related information together, such as student details (name, roll number, marks).

Declaration of Structure

A structure is declared using the struct keyword followed by the structure name and its members.

Example:

```
struct Student
{
   int roll;
   char name[20];
   float marks;
};
```

Here, Student is the structure name, and roll, name, and marks are structure members.

Initialization of Structure

Structure members can be initialized at the time of declaration.

Example:

```
struct Student s1 = {1, "Rajvi", 85.5};
```

Here:

roll = 1

name = "Rajvi"

marks = 85.5

Accessing Structure Members

Structure members are accessed using the dot (.) operator.

Example:

```
printf("%d", s1.roll);
printf("%s", s1.name);
printf("%f", s1.marks);
```

The dot operator connects the structure variable with its members

13) Explain the importance of file handling in C. Discuss how to perform file operations like opening, closing, reading, and writing files.

File handling in C is used to store data permanently in files instead of keeping it only in memory. Normally, data stored in variables is lost when the program ends. File handling allows data to be saved on disk so that it can be used later.

File handling is important because it helps in storing large amounts of data, maintaining records, and performing operations like reading, writing, and updating data. It is widely used in applications such as student record systems, banking systems, and inventory management systems.

File Operations in C

File operations in C are performed using functions from the stdio.h library. A file is handled using a file pointer.

1. Opening a File

A file is opened using the fopen() function. It requires the file name and the mode.

Common modes:

"r" → Read mode

"w" → Write mode

"a" → Append mode

Example:

```
FILE *fp;
```

```
fp = fopen("data.txt", "w");
```

This opens the file data.txt in write mode.

2. Writing to a File

Data can be written to a file using functions like fprintf() or fputs().

Example:

```
fprintf(fp, "Hello World");
```

This writes "Hello World" into the file.

3. Reading from a File

Data can be read from a file using functions like fscanf() or fgets().

Example:

```
char str[20];
fgets(str, 20, fp);
```

This reads data from the file into the string variable.

4. Closing a File

After performing operations, the file must be closed using fclose().

Example:

```
fclose(fp);
```

Closing the file ensures that data is properly saved and resources are released.